Listing 13-3

Command.java

```
public interface Command
{
  public void execute() throws Exception;
}
```

This may not seem very impressive. But imagine what would happen if one of the `Command` objects in the linked list cloned itself and then put the clone back on the list. The list would never go empty, and the `run()` function would never return.

Consider the test case in Listing 13-4. It creates something called a `SleepCommand`. Among other things, it passes a delay of 1000 ms to the constructor of the `SleepCommand`. It then puts the `SleepCommand` into the `ActiveObjectEngine`. After calling `run()`, it expects that a certain number of milliseconds has elapsed.

Listing 13-4

TestSleepCommand.java

```
import junit.framework.*;
import junit.swingui.TestRunner;

public class TestSleepCommand extends TestCase
{
  public static void main(String[] args)
  {
    TestRunner.main(new String[]{"TestSleepCommand"});
  }

  public TestSleepCommand(String name)
  {
    super(name);
  }

  private boolean commandExecuted = false;

  public void testSleep() throws Exception
  {
    Command wakeup = new Command()
    {
      public void execute() {commandExecuted = true;}
    };
    ActiveObjectEngine e = new ActiveObjectEngine();
    SleepCommand c = new SleepCommand(1000,e,wakeup);
    e.addCommand(c);
    long start = System.currentTimeMillis();
    e.run();
    long stop = System.currentTimeMillis();
    long sleepTime = (stop-start);
    assert("SleepTime " + sleepTime + " expected > 1000",
           sleepTime > 1000);
    assert("SleepTime " + sleepTime + " expected < 1100",
            sleepTime < 1100);
    assert("Command Executed", commandExecuted);
  }
}
```

Let's look at this test case more closely. The constructor of the `SleepCommand` contains three arguments. The first is the delay time in milliseconds. The second is the `ActiveObjectEngine` that the command will be running in. Finally, there is another command object called `wakeup`. The intent is that the `SleepCommand` will wait for the specified number of milliseconds and will then execute the `wakeup` command.

Listing 13-5 shows the implementation of `SleepCommand`. On execution, `SleepCommand` checks to see if it has been executed previously. If not, then it records the start time. If the delay time has not passed, it puts itself back in the `ActiveObjectEngine`. If the delay time has passed, it puts the `wakeup` command into the `ActiveObjectEngine`.

Listing 13-5

`SleepCommand.java`

```
public class SleepCommand implements Command
{
  private Command wakeupCommand = null;
  private ActiveObjectEngine engine = null;
  private long sleepTime = 0;
  private long startTime = 0;
  private boolean started = false;

  public SleepCommand(long milliseconds, ActiveObjectEngine e,
                      Command wakeupCommand)
  {
    sleepTime = milliseconds;
    engine = e;
    this.wakeupCommand = wakeupCommand;
  }

  public void execute() throws Exception
  {
    long currentTime = System.currentTimeMillis();
    if (!started)
    {
      started = true;
      startTime = currentTime;
      engine.addCommand(this);
    }
    else if ((currentTime - startTime) < sleepTime)
    {
      engine.addCommand(this);
    }
    else
    {
      engine.addCommand(wakeupCommand);
    }
  }
}
```

We can draw an analogy between this program and a multithreaded program that is waiting for an event. When a thread in a multithreaded program waits for an event, it usually invokes some operating system call that blocks the thread until the event has occurred. The program in Listing 13-5 does not block. Instead, if the event it is waiting for `((currentTime - startTime) < sleepTime)` has not occurred, it simply puts itself back into the `ActiveObjectEngine`.

Building multithreaded systems using variations of this technique has been, and will continue to be, a very common practice. Threads of this kind have been known as *run-to-completion* tasks (RTC), because each `Command` instance runs to completion before the next `Command` instance can run. The name RTC implies that the `Command` instances do not block.

The fact that the `Command` instances all run to completion gives RTC threads the interesting advantage that they all share the same run-time stack. Unlike the threads in a traditional multithreaded system, it is not necessary to define or allocate a separate run-time stack for each RTC thread. This can be a powerful advantage in memory-constrained systems with many threads.

Continuing our example, Listing 13-6 shows a simple program that makes use of `SleepCommand` and exhibits multithreaded behavior. This program is called `DelayedTyper`.

Listing 13-6

DelayedTyper.java

```
public class DelayedTyper implements Command
{
  private long itsDelay;
  private char itsChar;
  private static ActiveObjectEngine engine =
    new ActiveObjectEngine();
  private static boolean stop = false;

  public static void main(String args[])
  {
    engine.addCommand(new DelayedTyper(100,'1'));
    engine.addCommand(new DelayedTyper(300,'3'));
    engine.addCommand(new DelayedTyper(500,'5'));
    engine.addCommand(new DelayedTyper(700,'7'));

    Command stopCommand = new Command()
    {
      public void execute() {stop=true;}
    };

    engine.addCommand(
      new SleepCommand(20000,engine,stopCommand));
    engine.run();
  }

  public DelayedTyper(long delay, char c)
  {
    itsDelay = delay;
    itsChar = c;
  }

  public void execute() throws Exception
  {
    System.out.print(itsChar);
    if (!stop)
      delayAndRepeat();
  }
```

```
  private void delayAndRepeat() throws Exception
  {
    engine.addCommand(new SleepCommand(itsDelay,engine,this);
  }
}
```

Notice that `DelayedTyper` implements `Command`. The `execute` method simply prints a character that was passed at construction, checks the `stop` flag, and, if not set, invokes `delayAndRepeat`. The `delayAndRepeat` method constructs a `SleepCommand`, using the delay that was passed in at construction. It then inserts the `SleepCommand` into the `ActiveObjectEngine`.

The behavior of this `Command` is easy to predict. In effect, it hangs in a loop repeatedly typing a specified character and waiting for a specified delay. It exits the loop when the `stop` flag is set.

The main program of `DelayedTyper` starts several `DelayedTyper` instances going in the `ActiveObjectEngine`, each with its own character and delay. It then invokes a `SleepCommand` that will set the `stop` flag after a while. Running this program produces a simple string of 1's, 3's, 5's and 7's. Running it again produces a similar, but different string. Here are two typical runs:

```
13571131151137111315113171513111315173111135111371153111357...
13571113151317113151131171351113115173111315113171135111317...
```

These strings are different because the CPU clock and the real-time clock aren't in perfect sync. This kind of nondeterministic behavior is the hallmark of multithreaded systems.

Nondeterministic behavior is also the source of much woe, anguish, and pain. As anyone who has worked on embedded real-time systems knows, it's tough to debug nondeterministic behavior.

Conclusion

The simplicity of the COMMAND pattern belies its versatility. COMMAND can be used for a wonderful variety of purposes ranging from database transactions, to device control, to multithreaded nuclei, to GUI do/undo administration.

It has been suggested that the COMMAND pattern breaks the OO paradigm because it emphasizes functions over classes. That may be true, but in the real world of the software developer, the COMMAND pattern can be very useful.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Lavender, R. G., and D. C. Schmidt. *Active Object: An Object Behavioral Pattern for Concurrent Programming*, in "Pattern Languages of Program Design" (J. O. Coplien, J. Vlissides, and N. Kerth, eds.). Reading, MA: Addison–Wesley, 1996.

14

TEMPLATE METHOD & STRATEGY: Inheritance vs. Delegation

"The best strategy in life is diligence."

—Chinese Proverb

Way, way back in the early 90s—back in the early days of OO—we were all quite taken with the notion of inheritance. The implications of the relationship were profound. With inheritance we could *program by difference*! That is, given some class that did something almost useful to us, we could create a subclass and change only the bits we didn't like. We could reuse code simply by inheriting it! We could establish whole taxonomies of software structures, each level of which reused code from the levels above. It was a brave new world.

Like most brave new worlds, this one turned out to be a bit too starry eyed. By 1995, it was clear that inheritance was very easy to overuse and that overuse of inheritance was very costly. Gamma, Helm, Johnson, and Vlissides went so far as to stress, "*Favor object composition over class inheritance.*"[1] So we cut back on our use of inheritance, often replacing it with composition or delegation.

This chapter is the story of two patterns that epitomize the difference between inheritance and delegation. TEMPLATE METHOD and STRATEGY solve similar problems and can often be used interchangeably. However, TEMPLATE METHOD uses inheritance to solve the problem, whereas STRATEGY uses delegation.

TEMPLATE METHOD and STRATEGY both solve the problem of separating a generic algorithm from a detailed context. We see the need for this very frequently in software design. We have an algorithm that is generically applicable. In order to conform to the Dependency-Inversion Principle (DIP), we want to make sure that the

1. [GOF95], p. 20.

generic algorithm does not depend on the detailed implementation. Rather, we want the generic algorithm and the detailed implementation to depend on abstractions.

TEMPLATE METHOD

Consider all the programs you have written. Many probably have this fundamental main-loop structure.

```
Initialize();
while (!done()) // main loop
{
  Idle();       // do something useful.
}
Cleanup();
```

First we initialize the application. Then we enter the main loop. In the main loop we do whatever the program needs us to do. We might process GUI events or perhaps process database records. Finally, once we are done, we exit the main loop and clean up before we exit.

This structure is so common that we can capture it in a class named `Application`. Then we can reuse that class for every new program we want to write. Think of it! We never have to write that loop again![2]

For example, consider Listing 14-1. Here we see all the elements of the standard program. The `InputStreamReader` and `BufferedReader` are initialized. There is a main loop that reads Fahrenheit readings from the `BufferedReader` and prints out Celsius conversions. At the end, an exit message is printed.

Listing 14-1

`ftoc raw`

```
import java.io.*;
public class ftocraw
{
  public static void main(String[] args) throws Exception
  {
    InputStreamReader isr = new InputStreamReader(System.in);
    BufferedReader br = new BufferedReader(isr);
    boolean done = false;
    while (!done)
    {
      String fahrString = br.readLine();
      if (fahrString == null || fahrString.length() == 0)
        done = true;
      else
      {
        double fahr = Double.parseDouble(fahrString);
        double celcius = 5.0/9.0*(fahr-32);
        System.out.println("F=" + fahr + ", C=" + celcius);
      }
    }
    System.out.println("ftoc exit");
  }
}
```

This program has all the elements of the main-loop structure. It does a little initialization, does its work in a main loop, and then cleans up and exits.

2. I've also got this bridge I'd like to sell you.

We can separate this fundamental structure from the `ftoc` program by employing the TEMPLATE METHOD pattern. This pattern places all the generic code into an implemented method of an abstract base class. The implemented method captures the generic algorithm, but defers all details to abstract methods of the base class.

So, for example, we can capture the main-loop structure in an abstract base class called `Application`. (See Listing 14-2.)

Listing 14-2

`Application.java`

```
public abstract class Application
{
  private boolean isDone = false;

  protected abstract void init();
  protected abstract void idle();
  protected abstract void cleanup();

  protected void setDone()
  {isDone = true;}

  protected boolean done()
  {return isDone;}

  public void run()
  {
    init();
    while (!done())
      idle();
    cleanup();
  }
}
```

This class describes a generic main-loop application. We can see the main loop in the implemented `run` function. We can also see that all the work is being deferred to the abstract methods `init`, `idle`, and `cleanup`. The `init` method takes care of any initialization we need done. The `idle` method does the main work of the program and will be called repeatedly until `setDone` is called. The `cleanup` method does whatever needs to be done before we exit.

We can rewrite the `ftoc` class by inheriting from `Application` and just filling in the abstract methods. Listing 14-3 show what this looks like.

Listing 14-3

`ftocTemplateMethod.java`

```
import java.io.*;
public class ftocTemplateMethod extends Application
{
  private InputStreamReader isr;
  private BufferedReader br;

  public static void main(String[] args) throws Exception
  {
    (new ftocTemplateMethod()).run();
  }
```

```
  protected void init()
  {
    isr = new InputStreamReader(System.in);
    br = new BufferedReader(isr);
  }

  protected void idle()
  {
    String fahrString = readLineAndReturnNullIfError();
    if (fahrString == null || fahrString.length() == 0)
      setDone();
    else
    {
      double fahr = Double.parseDouble(fahrString);
      double celcius = 5.0/9.0*(fahr-32);
      System.out.println("F=" + fahr + ", C=" + celcius);
    }
  }

  protected void cleanup()
  {
    System.out.println("ftoc exit");
  }

  private String readLineAndReturnNullIfError()
  {
    String s;
    try
    {
      s = br.readLine();
    }
    catch(IOException e)
    {
      s = null;
    }
    return s;
  }
}
```

Dealing with the exception made the code get a little longer, but it's easy to see how the old `ftoc` application has been fit into the TEMPLATE METHOD pattern.

Pattern Abuse

By this time you should be thinking, "*Is he serious? Does he really expect me to use this* `Application` *class for all new apps? It hasn't bought me anything, and it's overcomplicated the problem.*"

I chose the example because it was simple and provided a good platform for showing the mechanics of TEMPLATE METHOD. On the other hand, I don't really recommend building `ftoc` like this.

This is a good example of pattern abuse. Using TEMPLATE METHOD for this particular application is ridiculous. It complicates the program and makes its bigger. Encapsulating the main loop of every application in the universe sounded wonderful when we started, but the practical application is fruitless in this case.

Design patterns are wonderful things. They can help you with many design problems. But the fact that they exist does not mean that they should always be used. In this case, while TEMPLATE METHOD was applicable to the problem, its use was not advisable. The cost of the pattern was higher than the benefit it yielded.

So let's look at a slightly more useful example. (See Listing 14-4.)

Bubble Sort[3]

Listing 14-4

BubbleSorter.java

```
public class BubbleSorter
{
  static int operations = 0;
  public static int sort(int [] array)
  {
    operations = 0;
    if (array.length <= 1)
      return operations;

    for (int nextToLast = array.length-2;
         nextToLast >= 0; nextToLast--)
      for (int index = 0; index <= nextToLast; index++)
        compareAndSwap(array, index);

    return operations;
  }

  private static void swap(int[] array, int index)
  {
    int temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  private static void compareAndSwap(int[] array, int index)
  {
    if (array[index] > array[index+1])
      swap(array, index);
    operations++;
  }
}
```

The `BubbleSorter` class knows how to sort an array of integers using the bubble-sort algorithm. The `sort` method of `BubbleSorter` contains the algorithm that knows how to do a bubble sort. The two ancillary methods, `swap` and `compareAndSwap`, deal with the details of integers and arrays and handle the mechanics that the `sort` algorithm requires.

3. Like `Application`, `Bubble Sort` is easy to understand and so makes a useful teaching tool. However, no one in their right mind would ever actually use a `Bubble Sort` if they had any significant amount of sorting to do. There are *much* better algorithms.

Using the TEMPLATE METHOD pattern, we can separate the bubble-sort algorithm out into an abstract base class named `BubbleSorter`. `BubbleSorter` contains an implementation of the `sort` function that calls an abstract method named `outOfOrder` and another called `swap`. The `outOfOrder` method compares two adjacent elements in the array and returns `true` if the elements are out of order. The `swap` method swaps two adjacent cells in the array.

The `sort` method does not know about the array, nor does it care what kind of objects are stored in the array. It just calls `outOfOrder` for various indices into the array and determines whether those indices should be swapped or not. (See Listing 14-5.)

Listing 14-5

BubbleSorter.java

```
public abstract class BubbleSorter
{
  private int operations = 0;
  protected int length = 0;

  protected int doSort()
  {
    operations = 0;
    if (length <= 1)
      return operations;

    for (int nextToLast = length-2;
         nextToLast >= 0; nextToLast--)
      for (int index = 0; index <= nextToLast; index++)
      {
        if (outOfOrder(index))
          swap(index);
        operations++;
      }

    return operations;
  }

  protected abstract void swap(int index);
  protected abstract boolean outOfOrder(int index);
}
```

Given `BubbleSorter` we can now create simple derivatives that can sort any different kind of object. For example, we could create `IntBubbleSorter`, which sorts arrays of integers, and `DoubleBubbleSorter`, which sorts arrays of doubles. (See Figure 14-1, Listing 14-6, and Listing 14-7.)

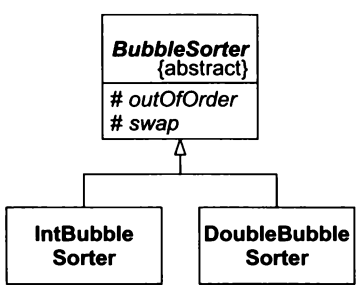


Figure 14-1 Bubble-Sorter Structure

Listing 14-6

IntBubbleSorter.java

```
public class IntBubbleSorter extends BubbleSorter
{
  private int[] array = null;
  public int sort(int [] theArray)
  {
    array = theArray;
    length = array.length;
    return doSort();
  }

  protected void swap(int index)
  {
    int temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  protected boolean outOfOrder(int index)
  {
    return (array[index] > array[index+1]);
  }
}
```

Listing 14-7

DoubleBubbleSorter.java

```
public class DoubleBubbleSorter extends BubbleSorter
{
  private double[] array = null;
  public int sort(double [] theArray)
  {
    array = theArray;
    length = array.length;
    return doSort();
  }

  protected void swap(int index)
  {
    double temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  protected boolean outOfOrder(int index)
  {
    return (array[index] > array[index+1]);
  }
}
```

The TEMPLATE METHOD pattern shows one of the classic forms of reuse in object-oriented programming. Generic algorithms are placed in the base class and inherited into different detailed contexts. But this technique is not without its costs. Inheritance is a very strong relationship. Derivatives are inextricably bound to their base classes.

For example, the `outOfOrder` and `swap` functions of `IntBubbleSorter` are exactly what are needed for other kinds of sort algorithms. And yet, there is no way to reuse `outOfOrder` and `swap` in those other sort algorithms. By inheriting `BubbleSorter`, we have doomed `IntBubbleSorter` to be bound forever to `BubbleSorter`. The STRATEGY pattern provides another option.

STRATEGY

The STRATEGY pattern solves the problem of inverting the dependencies of the generic algorithm and the detailed implementation in a very different way. Consider, once again, the pattern-abusing `Application` problem.

Rather than placing the generic application algorithm into an abstract base class, we place it into a *concrete* class named `ApplicationRunner`. We define the abstract methods that the generic algorithm must call within an interface named `Application`. We derive `ftocStrategy` from this interface and pass it into the `ApplicationRunner`. `ApplicationRunner` then delegates to this interface. (See Figure 14-2, and Listings 14-8 through 14-10.)

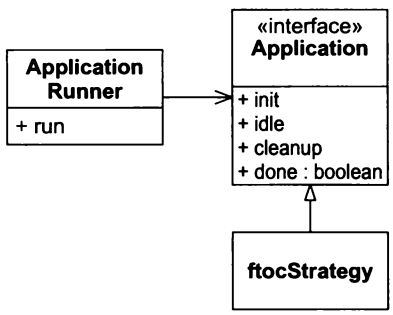


Figure 14-2 Strategy of the Application algorithm

Listing 14-8

ApplicationRunner.java

```
public class ApplicationRunner
{
  private Application itsApplication = null;

  public ApplicationRunner(Application app)
  {
    itsApplication = app;
  }
  public void run()
  {
    itsApplication.init();
    while (!itsApplication.done())
      itsApplication.idle();
    itsApplication.cleanup();
  }
}
```

Listing 14-9

Application.java

```
public interface Application
{
  public void init();
  public void idle();
```

```
  public void cleanup();
  public boolean done();
}
```

Listing 14-10

ftocStrategy.java

```
import java.io.*;
public class ftocStrategy implements Application
{
  private InputStreamReader isr;
  private BufferedReader br;
  private boolean isDone = false;

  public static void main(String[] args) throws Exception
  {
    (new ApplicationRunner(new ftocStrategy())).run();
  }

  public void init()
  {
    isr = new InputStreamReader(System.in);
    br = new BufferedReader(isr);
  }

  public void idle()
  {
    String fahrString = readLineAndReturnNullIfError();
    if (fahrString == null || fahrString.length() == 0)
      isDone = true;
    else
    {
      double fahr = Double.parseDouble(fahrString);
      double celcius = 5.0/9.0*(fahr-32);
      System.out.println("F=" + fahr + ", C=" + celcius);
    }
  }

  public void cleanup()
  {
    System.out.println("ftoc exit");
  }

  public boolean done()
  {
    return isDone;
  }

  private String readLineAndReturnNullIfError()
  {
    String s;
    try
    {
      s = br.readLine();
    }
    catch(IOException e)
```

```
    {
      s = null;
    }
    return s;
  }
}
```

It should be clear that this structure has both benefits and costs over the TEMPLATE METHOD structure. STRATEGY involves more total classes and more indirection than TEMPLATE METHOD. The delegation pointer within `ApplicationRunner` incurs a slightly higher cost in terms of run time and data space than inheritance would. On the other hand, if we had many different applications to run, we could reuse the `ApplicationRunner` *instance* and pass in many different implementations of `Application`, thereby reducing the coupling between the generic algorithm and the details it controls.

None of these costs and benefits are overriding. In most cases, none of them matters in the slightest. In the typical case, the most worrisome is the extra class needed by the STRATEGY pattern. However, there is more to consider.

Sorting Again

Consider implementating the bubble sort using the STRATEGY pattern. (See Listings 14-11 through 14-13.)

Listing 14-11

`BubbleSorter.java`

```
public class BubbleSorter
{
  private int operations = 0;
  private int length = 0;
  private SortHandle itsSortHandle = null;

  public BubbleSorter(SortHandle handle)
  {
    itsSortHandle = handle;
  }

  public int sort(Object array)
  {
    itsSortHandle.setArray(array);
    length = itsSortHandle.length();
    operations = 0;
    if (length <= 1)
      return operations;

    for (int nextToLast = length-2;
         nextToLast >= 0; nextToLast--)
      for (int index = 0; index <= nextToLast; index++)
      {
        if (itsSortHandle.outOfOrder(index))
          itsSortHandle.swap(index);
        operations++;
      }

    return operations;
  }
}
```

Listing 14-12

SortHandle.java

```
public interface SortHandle
{
  public void swap(int index);
  public boolean outOfOrder(int index);
  public int length();
  public void setArray(Object array);
}
```

Listing 14-13

IntSortHandle.java

```
public class IntSortHandle implements SortHandle
{
  private int[] array = null;

  public void swap(int index)
  {
    int temp = array[index];
    array[index] = array[index+1];
    array[index+1] = temp;
  }

  public void setArray(Object array)
  {
    this.array = (int[])array;
  }

  public int length()
  {
    return array.length;
  }

  public boolean outOfOrder(int index)
  {
    return (array[index] > array[index+1]);
  }
}
```

Notice that the `IntSortHandle` class knows nothing of the `BubbleSorter`. It has no dependency whatever on the bubble-sort implementation. This is not the case with the TEMPLATE METHOD pattern. Look back at Listing 14-6, and you can see that the `IntBubbleSorter` depended directly on `BubbleSorter`, the class that contains the bubble-sort algorithm.

The TEMPLATE METHOD approach partially violates DIP by Implementating the `swap` and `outOfOrder` methods to depend directly on the bubble-sort algorithm. The STRATEGY approach contains no such dependency. Thus, we can use the `IntSortHandle` with `Sorter` implementations other than `BubbleSorter`.

For example, we can create a variation of the bubble sort that terminates early if a pass through the array finds it in order. (See Listing 14-14.) `QuickBubbleSorter` can also use `IntSortHandle` or any other class derived from `SortHandle`.

Listing 14-14

QuickBubbleSorter.java

```
public class QuickBubbleSorter
{
  private int operations = 0;
  private int length = 0;
  private SortHandle itsSortHandle = null;

  public QuickBubbleSorter(SortHandle handle)
  {
    itsSortHandle = handle;
  }

  public int sort(Object array)
  {
    itsSortHandle.setArray(array);
    length = itsSortHandle.length();
    operations = 0;
    if (length <= 1)
      return operations;

    boolean thisPassInOrder = false;
    for (int nextToLast = length-2; nextToLast >= 0 && !thisPassInOrder; nextToLast--)
    {
      thisPassInOrder = true; //potenially.
      for (int index = 0; index <= nextToLast; index++)
      {
        if (itsSortHandle.outOfOrder(index))
        {
          itsSortHandle.swap(index);
          thisPassInOrder = false;
        }
        operations++;
      }
    }

    return operations;
  }
}
```

Thus, the STRATEGY pattern provides one extra benefit over the TEMPLATE METHOD pattern. Whereas the TEMPLATE METHOD pattern allows a generic algorithm to manipulate many possible detailed implementations, the STRATEGY pattern by fully conforming to the DIP allows each detailed implementation to be manipulated by many different generic algorithms.

Conclusion

Both TEMPLATE METHOD and STRATEGY allow you to separate high-level algorithms from low-level details. Both allow the high-level algorithms to be reused independently of the details. At the cost of a little extra complexity, memory, and runtime, STRATEGY also allows the details to be reused independently of the high-level algorithm.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.

15

FACADE and MEDIATOR

Symbolism erects a facade of respectability to hide the indecency of dreams.

—Mason Cooley

The two patterns discussed in this chapter have a common purpose. Both impose some kind of policy on another group of objects. FACADE imposes policy from above, and MEDIATOR imposes policy from below. The use of FACADE is visible and constraining, while the use of MEDIATOR is invisible and enabling.

FACADE

The FACADE pattern is used when you want to provide a simple and specific interface onto a group of objects that has a complex and general interface. Consider, for example, `DB.java` in Listing 26-9 on page 333. This class imposes a very simple interface, specific to `ProductData`, on the complex and general interfaces of the classes within the `java.sql` package. Figure 15-1 shows the structure.

Notice that the DB class protects the `Application` from needing to know the intimacies of the `java.sql` package. It hides all the generality and complexity of `java.sql` behind a very simple and specific interface.

A FACADE like DB imposes a lot of policy on the usage of the `java.sql` package. It knows how to initialize and close the database connection. It knows how to translate the members of `ProductData` into database fields and back. It knows how to build the appropriate queries and commands to manipulate the database. And it hides all that complexity from its users. From the point of view of the `Application`, `java.sql` does not exist; it is hidden behind the FACADE.

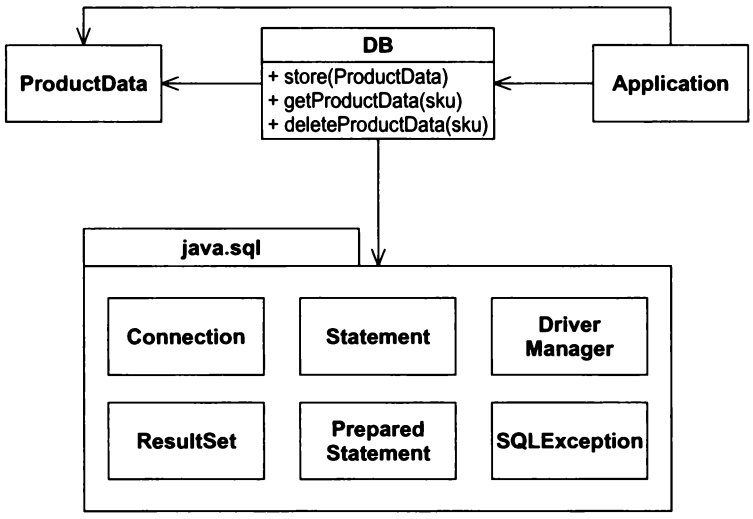


Figure 15-1 The DB FACADE

The use of the FACADE pattern implies that the developers have adopted the convention that all database calls must go through DB. If any part of the `Application` code goes straight to `java.sql` rather than through the FACADE, then that convention is violated. As such, the FACADE imposes its polices on the application. By convention, DB has become the sole broker of the facilities of `java.sql`.

MEDIATOR

The MEDIATOR pattern also imposes policy. However, whereas Facade imposed its policy in a visible and constraining way, MEDIATOR imposes its policies in a hidden and unconstrained way. For example, the `QuickEntryMediator` class in Listing 15–1 is a class that sits quietly behind the scenes and binds a text-entry field to a list. When you type in the text-entry field, the first element of the list that matches what you have typed is highlighted. This lets you type abbreviations and quickly select a list item.

Listing 15-1

`QuickEntryMediator.java`

```
package utility;

import javax.swing.*;
import javax.swing.event.*;

/**
QuickEntryMediator.  This class takes a JTextField and a JList.
It assumes that the user will type characters into the
JTextField that are prefixes of entries in the JList.  It
automatically selects the first item in the JList that matches
the current prefix in the JTextField.

If the JTextField is null, or the prefix does not match any
element in the JList, then the JList selection is cleared.

There are no methods to call for this object.  You simply
create it, and forget it.  (But don't let it be garbage
collected...)
```

Example:

```
JTextField t = new JTextField();
JList l = new JList();

QuickEntryMediator qem = new QuickEntryMediator(t, l);
 // that's all folks.

 @author Robert C. Martin, Robert S. Koss
 @date 30 Jun, 1999 2113 (SLAC)
 */

public class QuickEntryMediator {
  public QuickEntryMediator(JTextField t, JList l) {
    itsTextField = t;
    itsList = l;

    itsTextField.getDocument().addDocumentListener(
      new DocumentListener() {
        public void changedUpdate(DocumentEvent e) {
          textFieldChanged();
        }

        public void insertUpdate(DocumentEvent e) {
          textFieldChanged();
        }

        public void removeUpdate(DocumentEvent e) {
          textFieldChanged();
        }
      } // new DocumentListener
    ); // addDocumentListener
  } // QuickEntryMediator()

  private void textFieldChanged() {
    String prefix = itsTextField.getText();

    if (prefix.length() == 0) {
      itsList.clearSelection();
      return;
      }

      ListModel m = itsList.getModel();
      boolean found = false;
      for (int i = 0; found == false && i < m.getSize(); i++) {
      Object o = m.getElementAt(i);
      String s = o.toString();
      if (s.startsWith(prefix)) {
      itsList.setSelectedValue(o, true);
      found = true;
      }
    }

    if (!found) {
      itsList.clearSelection();
    }
  } // textFieldChanged
```

```
  private JTextField itsTextField;
  private JList itsList;
} // class QuickEntryMediator
```

The structure of the `QuickEntryMediator` is shown in Figure 15-2. An instance of `QuickEntryMediator` is constructed with a `JList` and a `JTextField`. The `QuickEntryMediator` registers an anonymous `DocumentListener` with the `JTextField`. This listener invokes the `textFieldChanged` method whenever there is a change in the text. This method then finds an element of the `JList` that is prefixed by the text and selects it.

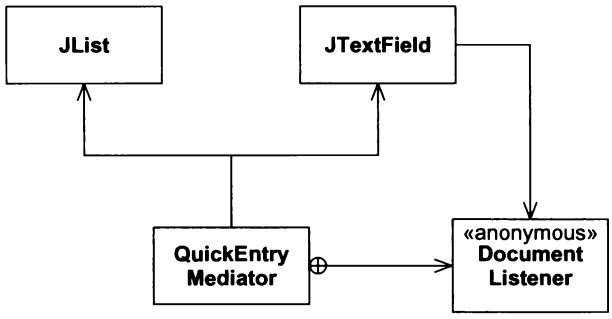


Figure 15-2 QuickEntryMediator

The users of the `JList` and `JTextField` have no idea that this MEDIATOR exists. It sits there quietly, imposing its policy on those objects without their permission or knowledge.

Conclusion

Imposing policy can be done from above using FACADE if that policy needs to be big and visible. On the other hand, if subtlety and discretion are needed, MEDIATOR may be the more appropriate choice. Facades are usually the focal point of a convention. Everyone agrees to use the facade instead of the objects beneath it. Mediator, on the other hand, is hidden from the users. Its policy is a fait accompli rather than a matter of convention.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.

16

SINGLETON and MONOSTATE

"Infinite beatitude of existence! It is; and there is none else beside It."

—The point. Flatland. Edwin A. Abbott

Usually there is a one-to-many relationship between classes and instances. You can create many instances of most classes. The instances are created when they are needed and disposed of when their usefulness ends. They come and go in a flow of memory allocations and deallocations.

However, there are some classes that should have only one instance. That instance should appear to have come into existence when the program started and should be disposed of only when the program ends. Such objects are sometimes the roots of the application. From the roots you can find your way to many other objects in the system. Sometimes they are factories, which you can use to create the other objects in the system. Sometimes these objects are managers, responsible for keeping track of certain other objects and driving them through their paces.

Whatever these objects are, it is a severe logic failure if more than one of them are created. If more than one root are created, then access to objects in the application may depend on a chosen root. Programmers, not knowing that more than one root exist, may find themselves looking at a subset of the application objects without knowing it. If more than one factory exist, clerical control over the created objects may be compromised. If more than one manager exist, activities that were intended to be serial may become concurrent.

It may seem that mechanisms to enforce the singularity of these objects are overkill. After all, when you initialize the application, you can simply create one of each and be done with it. In fact, this is usually the best course of action. Mechanism should be avoided when there is no immediate and significant need. However, we also want our code to communicate our intent. If the mechanism for enforcing singularity is trivial, the benefit of communication may outweigh the cost of the mechanism.

This chapter is about two patterns that enforce singularity. These patterns have very different cost–benefit trade-offs. In many contexts, their cost is low enough to more than balance the benefit of their expressiveness.

SINGLETON[1]

SINGLETON is a very simple pattern. The test case in Listing 16-1 shows how it should work. The first test function shows that the `Singleton` instance is accessed through the public static method `Instance`. It also shows that if `Instance` is called multiple times, a reference to the exact same instance is returned each time. The second test case shows that the `Singleton` class has no public constructors, so there is no way for anyone to create an instance without using the `Instance` method.

Listing 16-1

Singleton Test Case

```
import junit.framework.*;
import java.lang.reflect.Constructor;

public class TestSimpleSingleton extends TestCase
{
  public TestSimpleSingleton(String name)
  {
    super(name);
  }

  public void testCreateSingleton()
  {
    Singleton s = Singleton.Instance();
    Singleton s2 = Singleton.Instance();
    assertSame(s, s2);
  }

  public void testNoPublicConstructors() throws Exception
  {
    Class singleton = Class.forName("Singleton");
    Constructor[] constructors = singleton.getConstructors();
    assertEquals("public constructors.",
                 0, constructors.length);
  }
}
```

This test case is a specification for the SINGLETON pattern. It leads directly to the code shown in Listing 16-2. It should be clear, by inspecting this code, that there can never be more than one instance of the `Singleton` class within the scope of the static variable `Singleton.theInstance`.

1. [GOF95], p. 127.

Listing 16-2

Singleton Implementation

```
public class Singleton
{
  private static Singleton theInstance = null;
  private Singleton() {}

  public static Singleton Instance()
  {
    if (theInstance == null)
      theInstance = new Singleton();
    return theInstance;
  }
}
```

Benefits of the SINGLETON

- **Cross platform.** Using appropriate middleware (e.g., RMI), SINGLETON can be extended to work across many JVMs and many computers.
- **Applicable to any class.** You can change any class into a SINGLETON simply by making its constructors private and by adding the appropriate static functions and variable.
- **Can be created through derivation.** Given a class, you can create a subclass that is a SINGLETON.
- **Lazy evaluation**. If the SINGLETON is never used, it is never created.

Costs of the SINGLETON

- **Destruction is undefined.** There is no good way to destroy or decommission a SINGLETON. If you add a `decommission` method that nulls out `theInstance`, other modules in the system may still be holding a reference to the SINGLETON instance. Subsequent calls to `Instance` will cause another instance to be created, causing two concurrent instances to exist. This problem is particularly acute in C++ where the instance *can be destroyed*, leading to possible dereferencing of a destroyed object.
- **Not inherited.** A class derived from a SINGLETON is not a singleton. If it needs to be a SINGLETON, the static function and variable need to be added to it.
- **Efficiency.** Each call to `Instance` invokes the `if` statement. For most of those calls, the `if` statement is useless.
- **Nontransparent.** Users of a SINGLETON know that they are using a SINGLETON because they must invoke the `Instance` method.

SINGLETON in Action

Assume that we have a Web-based system that allows users to log in to secure areas of a Web server. Such a system will have a database containing user names, passwords, and other user attributes. Assume further that the database is accessed through a third-party API. We could access the database directly in every module that needed to read and write a user. However, this would scatter usage of the third-party API throughout the code, and it would leave us no place to enforce access or structure conventions.

A better solution is to use the FACADE pattern and create a `UserDatabase` class that provides methods for reading and writing `User` objects. These methods access the third-party API of the database, translating between `User` objects and the tables and rows of the database. Within the `UserDatabase`, we can enforce conventions of structure and access. For example, we can guarantee that no `User` record gets written unless it has a nonblank `username`. Or we can serialize access to a `User` record, making sure that two modules cannot simultaneously read and write it.

The code in Listings 16-3 and 16-4 shows a SINGLETON solution. The SINGLETON class is named `UserDatabaseSource`. It implements the `UserDatabase` interface. Notice that the static `instance()` method does not have the traditional `if` statement to protect against multiple creations. Instead, it takes advantage of the Java initialization facility.

Listing 16-3

`UserDatabase Interface`

```
public interface UserDatabase
{
  User readUser(String userName);
  void writeUser(User user);
}
```

Listing 16-4

`UserDatabaseSource Singleton`

```
public class UserDatabaseSource implements UserDatabase
{
  private static UserDatabase theInstance =
    new UserDatabaseSource();

  public static UserDatabase instance()
  {
    return theInstance;
  }

  private UserDatabaseSource()
  {
  }

  public User readUser(String userName)
  {
    // Some Implementation
    return null;  // just to make it compile.
  }

  public void writeUser(User user)
  {
    // Some Implementation
  }
}
```

This is an extremely common use of the SINGLETON pattern. It assures that all database access will be through a single instance of `UserDatabaseSource`. This makes it easy to put checks, counters, and locks in `UserDatabaseSource` that enforce the access and structure conventions mentioned earlier.

MONOSTATE[2]

The MONOSTATE pattern is another way to achieve singularity. It works through a completely different mechanism. We can see how that mechanism works by studying the MONOSTATE test case in Listing 16-5.

The first test function simply describes an object whose `x` variable can be set and retrieved. But the second test case shows that two instances of the same class behave *as though they were one*. If you set the `x` variable on

2. [BALL2000].

one instance to a particular value, you can retrieve that value by getting the x variable of a different instance. It's as though the two instances are just different names for the same object.

Listing 16-5

Monostate Test Case

```
import junit.framework.*;

public class TestMonostate extends TestCase
{
  public TestMonostate(String name)
  {
    super(name);
  }

  public void testInstance()
  {
    Monostate m = new Monostate();
    for (int x = 0; x<10; x++)
    {
      m.setX(x);
      assertEquals(x, m.getX());
    }
  }

  public void testInstancesBehaveAsOne()
  {
    Monostate m1 = new Monostate();
    Monostate m2 = new Monostate();

    for (int x = 0; x<10; x++)
    {
      m1.setX(x);
      assertEquals(x, m2.getX());
    }
  }
}
```

If we were to plug the `Singleton` class into this test case and replace all the `new Monostate` statements with calls to `Singleton.Instance`, the test case should still pass. So this test case describes the *behavior* of `Singleton` without imposing the constraint of a single instance!

How can two instances behave as though they were a single object? Quite simply it means that the two objects must share the same variables. This is easily achieved by making all the variables static. Listing 16-6 shows the implementation of `Monostate` that passes the above test case. Note that the `itsX` variable is static. Note also that *none of the methods are static*. This is important as we'll see later.

Listing 16-6

Monostate Implementation

```
public class Monostate
{
  private static int itsX = 0;
  public Monostate() {}

  public void setX(int x)
```

```
  {
    itsX = x;
  }

  public int getX()
  {
    return itsX;
  }
}
```

I find this to be a delightfully twisted pattern. No matter how many instances of `Monostate` you create, they all behave as though they were a *single object*. You can even destroy or decommission all the current instances without losing the data.

Note that the difference between the two patterns is one of behavior vs. structure. The SINGLETON pattern enforces the structure of singularity. It prevents any more than one instance from being created. Whereas MONOSTATE enforces the *behavior* of singularity without imposing structural constraints. To underscore this difference consider that the MONOSTATE test case is valid for the `Singleton` class, but the SINGLETON test case is not even close to being valid for the `Monostate` class.

Benefits of MONOSTATE

- **Transparency**. Users of a MONOSTATE do not behave differently than users of a regular object. The users do not need to know that the object is MONOSTATE.
- **Derivability**. Derivatives of a MONOSTATE are MONOSTATES. Indeed, all the derivatives of a MONOSTATE are part of the *same* MONOSTATE. They all share the same static variables.
- **Polymorphism**. Since the methods of a MONOSTATE are not static, they can be overridden in a derivative. Thus different derivatives can offer different behavior over the same set of static variables.
- **Well-defined creation and destruction.** The variables of a MONOSTATE, being static, have well-defined creation and destruction times.

Costs of MONOSTATE

- **No conversion**. A normal class cannot be converted into a MONOSTATE class through derivation.
- **Efficiency**. A MONOSTATE may go through many creations and destructions because it is a real object. These operations are often costly.
- **Presence**. The variables of a MONOSTATE take up space, even if the MONOSTATE is never used.
- **Platform local.** You can't make a MONOSTATE work across several JVM instances or across several platforms.

MONOSTATE in Action

Consider implementing the simple finite state machine for a subway turnstile shown in Figure 16-1. The turnstile begins its life in the `Locked` state. If a coin is deposited, it transitions to the `Unlocked` state, unlocks the gate, resets any alarm state that might be present, and deposits the coin in its collection bin. If a user passes through the gate at this point, the turnstile transitions back to the `Locked` state and locks the gate.

There are two abnormal conditions. If the user deposits two or more coins before passing through the gate, the coins will be refunded and the gate will remain unlocked. If the user passes through without paying, then an alarm will sound and the gate will remain locked.

The test program that describes this operation is shown in Listing 16-7. Note that the test methods assume that the `Turnstile` is a monostate. It expects to be able to send events and gather queries from different instances. This makes sense if there will never be more than one instance of the `Turnstile`.

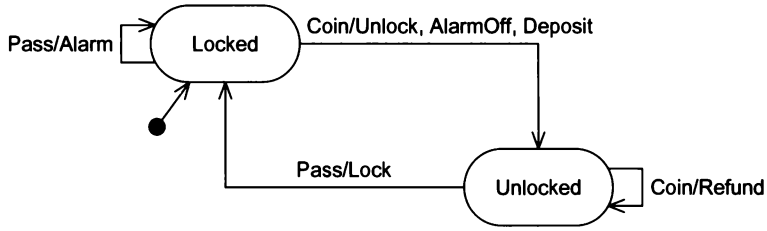


Figure 16-1 Subway Turnstile Finite State Machine

Listing 16-7

TestTurnstile

```
import junit.framework.*;

public class TestTurnstile extends TestCase
{
  public TestTurnstile(String name)
  {
    super(name);
  }

  public void setUp()
  {
    Turnstile t = new Turnstile();
    t.reset();
  }

  public void testInit()
  {
    Turnstile t = new Turnstile();
    assert(t.locked());
    assert(!t.alarm());
  }

  public void testCoin()
  {
    Turnstile t = new Turnstile();
    t.coin();
    Turnstile t1 = new Turnstile();
    assert(!t1.locked());
    assert(!t1.alarm());
    assertEquals(1, t1.coins());
  }

  public void testCoinAndPass()
  {
    Turnstile t = new Turnstile();
    t.coin();
    t.pass();

    Turnstile t1 = new Turnstile();
    assert(t1.locked());
    assert(!t1.alarm());
    assertEquals("coins", 1, t1.coins());
  }
```

```
  public void testTwoCoins()
  {
    Turnstile t = new Turnstile();
    t.coin();
    t.coin();

    Turnstile t1 = new Turnstile();
    assert("unlocked", !t1.locked());
    assertEquals("coins",1, t1.coins());
    assertEquals("refunds", 1, t1.refunds());
    assert(!t1.alarm());
  }

  public void testPass()
  {
    Turnstile t = new Turnstile();
    t.pass();
    Turnstile t1 = new Turnstile();
    assert("alarm", t1.alarm());
    assert("locked", t1.locked());
  }

  public void testCancelAlarm()
  {
    Turnstile t = new Turnstile();
    t.pass();
    t.coin();
    Turnstile t1 = new Turnstile();
    assert("alarm", !t1.alarm());
    assert("locked", !t1.locked());
    assertEquals("coin", 1, t1.coins());
    assertEquals("refund", 0, t1.refunds());
  }

  public void testTwoOperations()
  {
    Turnstile t = new Turnstile();
    t.coin();
    t.pass();
    t.coin();
    assert("unlocked", !t.locked());
    assertEquals("coins", 2, t.coins());
    t.pass();
    assert("locked", t.locked());
  }
}
```

The implementation of the monostate `Turnstile` is in Listing 16-8. The base `Turnstile` class delegates the two event functions (`coin` and `pass`) to two derivatives of `Turnstile` (`Locked` and `Unlocked`) that represent the states of the finite-state machine.

Listing 16-8

Turnstile

```
public class Turnstile
{
  private static boolean isLocked = true;
  private static boolean isAlarming = false;
  private static int itsCoins = 0;
  private static int itsRefunds = 0;
  protected final static Turnstile LOCKED = new Locked();
  protected final static Turnstile UNLOCKED = new Unlocked();
  protected static Turnstile itsState = LOCKED;

  public void reset()
  {
    lock(true);
    alarm(false);
    itsCoins = 0;
    itsRefunds = 0;
    itsState = LOCKED;
  }

  public boolean locked()
  {
    return isLocked;
  }

  public boolean alarm()
  {
    return isAlarming;
  }

  public void coin()
  {
    itsState.coin();
  }

  public void pass()
  {
    itsState.pass();
  }

  protected void lock(boolean shouldLock)
  {
    isLocked = shouldLock;
  }

  protected void alarm(boolean shouldAlarm)
  {
    isAlarming = shouldAlarm;
  }

  public int coins()
  {
    return itsCoins;
  }
```

```
  public int refunds()
  {
    return itsRefunds;
  }

  public void deposit()
  {
    itsCoins++;
  }

  public void refund()
  {
    itsRefunds++;
  }
}

class Locked extends Turnstile
{
  public void coin()
  {
    itsState = UNLOCKED;
    lock(false);
    alarm(false);
    deposit();
  }

  public void pass()
  {
    alarm(true);
  }
}


class Unlocked extends Turnstile
{
  public void coin()
  {
    refund();
  }

  public void pass()
  {
    lock(true);
    itsState = LOCKED;
  }
}
```

This example shows some of the useful features of the MONOSTATE pattern. It takes advantage of the ability for MONOSTATE derivatives to be polymorphic and the fact that MONOSTATE derivatives are themselves MONOSTATES. This example also shows how difficult it can sometimes be to turn a MONOSTATE into a normal class. The structure of this solution depends strongly on the MONOSTATE nature of `Turnstile`. If we needed to control more than one turnstile with this finite-state machine, the code would require some significant refactoring.

Perhaps you are concerned about the unconventional use of inheritance in this example. Having `Unlocked` and `Locked` derived from `Turnstile` seems a violation of normal OO principles. However, since `Turnstile` is

a MONOSTATE, there are no separate instances of it. Thus, `Unlocked` and `Locked` aren't really separate objects. Instead they are part of the `Turnstile` abstraction. `Unlocked` and `Locked` have access to the same variables and methods that `Turnstile` has access to.

Conclusion

It is often necessary to enforce the constraint that a particular object have only a single instantiation. This chapter has shown two very different techniques. SINGLETON makes use of private constructors, a static variable, and a static function to control and limit instantiation. MONOSTATE simply makes all variables of the object static.

SINGLETON is best used when you have an existing class that you want to constrain through derivation, and you don't mind that everyone will have to call the `instance()` method to gain access. MONOSTATE is best used when you want the singular nature of the class to be transparent to the users or when you want to employ polymorphic derivatives of the single object.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.
3. Ball, Steve, and John Crawford. Monostate Classes: The Power of One. Published in *More C++ Gems*, compiled by Robert C. Martin. Cambridge, UK: Cambridge University Press, 2000, p. 223.

17

NULL OBJECT

Faultily faultless, icily regular, splendidly null, Dead perfection, no more.

—Alfred Tennyson (1809–1892)

Consider the following code:

```
Employee e = DB.getEmployee("Bob");
if (e != null && e.isTimeToPay(today))
  e.pay();
```

We ask the database for an `Employee` object named "Bob." The `DB` object returns `null` if no such object exists. Otherwise, it returns the requested instance of `Employee`. If the employee exists, and if it is time to pay him, then we invoke the `pay` method.

We've all written code like this before. The idiom is common because the first expression of the `&&` is evaluated first in C-based languages, and the second is evaluated only if the first is `true`. Most of us have also been burned by forgetting to test for `null`. Common though the idiom may be, it is ugly and error prone.

We can alleviate the tendency toward error by having `DB.getEmployee` throw an exception instead of returning `null`. However, `try/catch` blocks can be even uglier than checking for `null`. Worse, the use of exceptions forces us to declare them in `throws` clauses. This makes it hard to retrofit exceptions into an existing application.

We can address these issues by using the NULL OBJECT pattern.[1] This pattern often eliminates the need to check for `null`, and it can help to simplify the code.

Figure 17-1 shows the structure. `Employee` becomes an interface that has two implementations. `EmployeeImplementation` is the normal implementation. It contains all the methods and variables that you would expect an `Employee` object to have. When `DB.getEmployee` finds an employee in the database, it returns an instance of `EmployeeImplementation`. `NullEmployee` is returned only if `DB.getEmployee` cannot find the employee.

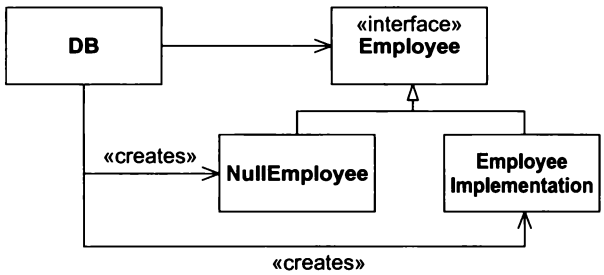


Figure 17-1 NULL OBJECT Pattern

`NullEmployee` implements all the methods of `Employee` to do "nothing." What "nothing" is depends on the method. For example, one would expect that `isTimeToPay` would be implemented to return `false`, since it is never time to pay a `NullEmployee`.

Using this pattern, we can change the original code to look like this:

```
Employee e = DB.getEmployee("Bob");
if (e.isTimeToPay(today))
  e.pay();
```

This is neither error prone nor ugly. There is a nice consistency to it. `DB.getEmployee` *always* returns an instance of `Employee`. That instance is guaranteed to behave appropriately, regardless of whether the employee was found or not.

Of course there will be many cases where we'll still want to know if `DB.getEmployee` failed to find an employee. This can be accomplished by creating a `static final` variable in `Employee` that holds the one and only instance of `NullEmployee`.

Listing 17-1 shows the test case for `NullEmployee`. In this case "Bob" does not exist in the database. Notice that the test case expects `isTimeToPay` to return `false`. Notice also that it expects the employee returned by `DB.getEmployee` to be `Employee.NULL`.

Listing 17-1

`TestEmployee.java (Partial)`

```
public void testNull() throws Exception
  {
    Employee e = DB.getEmployee("Bob");
```

1. [PLOPD3], p. 5. This delightful article, by Bobby Woolf, is full of wit, irony and practical advice.

```
    if (e.isTimeToPay(new Date()))
      fail();
    assertEquals(Employee.NULL, e);
  }
```

The DB class is shown in Listing 17-2. Notice that, for the purposes of our test, the `getEmployee` method just returns `Employee.NULL`.

Listing 17-2

DB.java

```
public class DB
{
  public static Employee getEmployee(String name)
  {
    return Employee.NULL;
  }
}
```

The `Employee` interface is shown in Listing 17-3. Notice that it has a `static` variable named `NULL` that holds an anonymous implementation of `Employee`. This anonymous implementation is the sole instance of the null employee. It implements `isTimeToPay` to return `false` and `pay` to do nothing.

Listing 17-3

Employee.java

```
import java.util.Date;
public interface Employee
{
  public boolean isTimeToPay(Date payDate);

  public void pay();

  public static final Employee NULL = new Employee()
  {
    public boolean isTimeToPay(Date payDate)
    {
      return false;
    }

    public void pay()
    {
    }
  };
}
```

Making the null employee an anonymous inner class is a way to make sure that there is only a single instance of it. There is no `NullEmployee` class per se. Nobody else can create other instances of the null employee. This is a good thing because we want to be able to say things like

```
if (e == Employee.NULL)
```

This would be unreliable if it were possible to create many instances of the null employee.

Conclusion

Those of us who have been using C-based languages for a long time have grown accustomed to functions that return `null` or `0` on some kind of failure. We presume that the return value from such functions needs to be tested. The NULL OBJECT pattern changes this. By using this pattern, we can ensure that functions always return valid objects, even when they fail. Those objects that represent failure do "nothing."

Bibliography

1. Martin, Robert, Dirk Riehle, and Frank Buschmann. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.

18

The Payroll Case Study: Iteration One Begins

"Everything which is in any way beautiful is beautiful in itself, and terminates in itself, not having praise as part of itself."

—Marcus Aurelius, circa A.D. 170

Introduction

The following case study describes the first iteration in the development of a simple batch payroll system. You will find the user stories in this case study to be simplistic. For example, taxes are simply not mentioned. This is typical of an early iteration. It will provide only a very small part of the business value the customers need.

In this chapter we will do the kind of quick analysis and design session that often takes place at the start of a normal iteration. The customer has selected the stories for the iteration, and now we have to figure out how we are going to implement them. Such design sessions are short and cursory, just like this chapter. The UML diagrams you see here are no more than hasty sketches on a whiteboard. The real design work will take place in the next chapter, when we work through the unit tests and implementations.

Specification

The following are some notes we took while conversing with our customer about the stories that were selected for the first iteration:

- Some employees work by the hour. They are paid an hourly rate that is one of the fields in their employee record. They submit daily time cards that record the date and the number of hours worked. If they work more than 8 hours per day, they are paid 1.5 times their normal rate for those extra hours. They are paid every Friday.
- Some employees are paid a flat salary. They are paid on the last working day of the month. Their monthly salary is one of the fields in their employee record.
- Some of the salaried employees are also paid a commission based on their sales. They submit sales receipts that record the date and the amount of the sale. Their commission rate is a field in their employee record. They are paid every other Friday.
- Employees can select their method of payment. They may have their paychecks mailed to the postal address of their choice; they may have their paychecks held for pickup by the paymaster; or they can request that their paychecks be directly deposited into the bank account of their choice.
- Some employees belong to the union. Their employee record has a field for the weekly dues rate. Their dues must be deducted from their pay. Also, the union may assess service charges against individual union members from time to time. These service charges are submitted by the union on a weekly basis and must be deducted from the appropriate employee's next pay amount.
- The payroll application will run once each working day and pay the appropriate employees on that day. The system will be told to what date the employees are to be paid, so it will generate payments for records from the last time the employee was paid up to the specified date.

We could begin by generating the database schema. Clearly this problem could use some kind of relational database, and the requirements give us a very good idea of what the tables and fields might be. It would be easy to design a workable schema and then start building some queries. However, this approach will generate an application for which the database is the central concern.

Databases are *implementation details*! Considering the database should be deferred as long as possible. Far too many applications are inextricably tied to their databases because they were designed with the database in mind from the beginning. Remember the definition of abstraction: *the amplification of the essential and the elimination of the irrelevant*. The database is irrelevant at this stage of the project; it is merely a technique used for storing and accessing data, nothing more.

Analysis by Use Cases

Instead of starting with the data of the system, let's start by considering the behavior of the system. After all, it is the system's behavior that we are being paid to create.

One way to capture and analyze the behavior of a system is to create *use cases*. Use cases, as originally described by Jacobson, are very similar to the notion of user stories in XP. A use case is like a user story that has been elaborated with a little more detail. Such elaboration is appropriate once the user story has been selected for implementation in the current iteration.

When we perform use case analysis, we look to the user stories and acceptance tests to find out the kinds of stimuli that the users of this system provide. Then we try to figure out how the system responds to those stimuli.

For example, here are the user stories that our customer has chosen for the next iteration:

1. Add a new employee
2. Delete an employee
3. Post a time card
4. Post a sales receipt
5. Post a union service charge
6. Change employee details (e.g., hourly rate, dues rate.)
7. Run the payroll for today

Let's convert each of these user stories into an elaborated use case. We don't need to go into too much detail—just enough to help us think through the design of the code that fulfills each story.

Adding Employees

Use Case 1
Add New Employee

A new employee is added by the receipt of an `AddEmp` transaction. This transaction contains the employee's name, address, and assigned employee number. The transaction has three forms:

```
AddEmp <EmpID> "<name>" "<address>" H <hourly-rate>
AddEmp <EmpID> "<name>" "<address>" S <monthly-salary>
AddEmp <EmpID> "<name>" "<address>" C <monthly-salary> <commission-rate>
```

The employee record is created with its fields assigned appropriately.

Alternative 1:
An error in the transaction structure

If the transaction structure is inappropriate, it is printed out in an error message, and no action is taken.

Use case 1 hints at an abstraction. There are three forms of the `AddEmp` transaction, yet all three forms share the `<EmpID>`, `<name>`, and `<address>` fields. We can use the COMMAND pattern to create an `AddEmployeeTransaction` abstract base class with three derivatives: `AddHourlyEmployeeTransaction`, `AddSalariedEmployeeTransaction` and `AddCommissionedEmployeeTransaction`. (See Figure 18-1.)

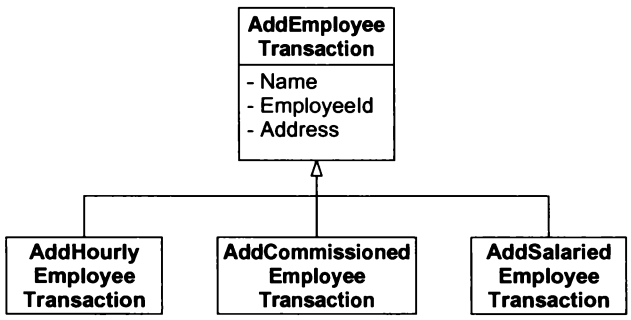


Figure 18-1 `AddEmployeeTransaction` Class Hierarchy

This structure conforms nicely to the Single-Responsibility Principle (SRP) by splitting each job into its own class. The alternative would be to put all these jobs into a single module. While this might reduce the number of classes in the system, and therefore make the system simpler, it would also concentrate all the transaction processing code in one place, creating a large and potentially error-prone module.

Use case 1 specifically talks about an employee record, which implies some sort of database. Again our predisposition to databases may tempt us into thinking about record layouts or the field structure in a relational database table, but we should resist these urges. What the use case is really asking us to do is to create an employee. What is the object model of an employee? A better question might be, What do the three different transactions create? In my view, they create three different kinds of employee objects, mimicking the three different kinds of `AddEmp` transactions. Figure 18-2 shows a possible structure.

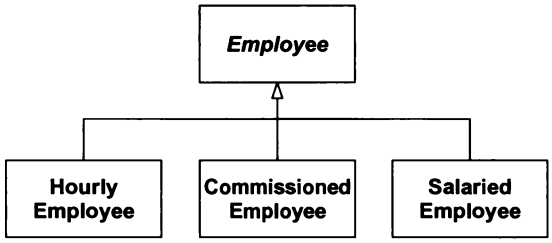


Figure 18-2 Possible `Employee` Class Hierarchy

Deleting Employees

Use Case 2
Deleting an Employee

Employees are deleted when a `DelEmp` transaction is received. The form of this transaction is as follows:

```
DelEmp <EmpID>
```

When this transaction is received, the appropriate employee record is deleted.

Alternative 1:
Invalid or unknown `EmpID`

If the `<EmpID>` field is not structured correctly, or if it does not refer to a valid employee record, then the transaction is printed with an error message, and no other action is taken.

This use case doesn't give me any design insights at this time, so let's look at the next.

Posting Time Cards

Use Case 3
Post a `Time Card`

On receiving a `TimeCard` transaction, the system will create a time-card record and associate it with the appropriate employee record.

```
TimeCard <Empld> <date> <hours>
```

Alternative 1:
The selected employee is not hourly

The system will print an appropriate error message and take no further action.

Alternative 2:
An error in the transaction structure

The system will print an appropriate error message and take no further action.

This use case points out that some transactions apply only to certain kinds of employees, strengthening the idea that the different kinds should be represented by different classes. In this case, there is also an association implied between time cards and hourly employees. Figure 18-3 shows a possible static model for this association.

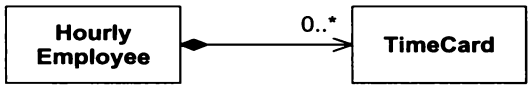


Figure 18-3 Association between `HourlyEmployee` and `TimeCard`

Posting Sales Receipts

Use Case 4
Posting a `Sales Receipt`

Upon receiving the `SalesReceipt` transaction, the system will create a new sales-receipt record and associate it with the appropriate commissioned employee.

```
SalesReceipt <EmpID> <date> <amount>
```

Alternative 1:
The selected employee is not commissioned

The system will print an appropriate error message and take no further action.

Alternative 2:
An error in the transaction structure

The system will print an appropriate error message and take no further action.

This use case is very similar to use case 3. It implies the structure shown in Figure 18-4.

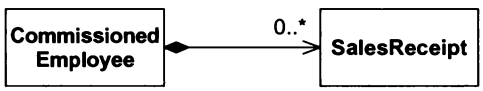


Figure 18-4 Commissioned Employees and Sales Receipts

Posting a Union Service Charge

Use Case 5
Posting a Union Service Charge

Upon receiving this transaction, the system will create a service-charge record and associate it with the appropriate union member.

```
ServiceCharge <memberID> <amount>
```

Alternative 1:
Poorly formed transaction

If the transaction is not well formed or if the `<memberID>` does not refer to an existing union member, then the transaction is printed with an appropriate error message.

This use case shows that union members are not accessed through employee IDs. The union maintains its own identification numbering scheme for union members. Thus, the system must be able to associate union

members and employees. There are many different ways to provide this kind of association, so to avoid being arbitrary, let's defer this decision until later. Perhaps constraints from other parts of the system will force our hand one way or another.

One thing is certain. There is a direct association between union members and their service charges. Figure 18-5 shows a possible static model for this association.

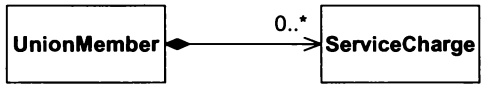


Figure 18-5 Union Members and Service Charges

Changing Employee Details

Use Case 6
Changing Employee Details

Upon receiving this transaction, the system will alter one of the details of the appropriate employee record. There are several possible variations to this transaction.

`ChgEmp <EmpID> Name <name>`	Change Employee Name
`ChgEmp <EmpID> Address <address>`	Change Employee Address
`ChgEmp <EmpID> Hourly <hourlyRate>`	Change to Hourly
`ChgEmp <EmpID> Salaried <salary>`	Change to Salaried
`ChgEmp <EmpID> Commissioned <salary> <rate>`	Change to Commissioned
`ChgEmp <EmpID> Hold`	Hold Paycheck
`ChgEmp <EmpID> Direct <bank> <account>`	Direct Deposit
`ChgEmp <EmpID> Mail <address>`	Mail Paycheck
`ChgEmp <EmpID> Member <memberID> Dues <rate>`	Put Employee in Union
`ChgEmp <EmpID> NoMember`	Remove Employee from Union

Alternative 1:
Transaction Errors

If the structure of the transaction is improper, or `<EmpID>` does not refer to a real employee, or `<memberID>` already refers to a member, then print a suitable error and take no further action.

This use case is very revealing. It has told us all the aspects of an employee that must be changeable. The fact that we can change an employee from hourly to salaried means that the diagram in Figure 18-2 is certainly invalid. Instead, it would probably be more appropriate to use the STRATEGY pattern for calculating pay. The `Employee` class could hold a strategy class named `PaymentClassification`, as in Figure 18-6. This is an advantage because we can change the `PaymentClassification` object without changing any other part of the `Employee` object. When an hourly employee is changed to a salaried employee, the `HourlyClassification` of the corresponding `Employee` object is replaced with a `SalariedClassification` object.

`PaymentClassification` objects come in three varieties. The `HourlyClassification` objects maintain the hourly rate and a list of `TimeCard` objects. The `SalariedClassification` objects maintain the monthly salary figure. The `CommissionedClassification` objects maintain a monthly salary, a commission rate, and a list of `SalesReceipt` objects. I have used composition relationships in these cases because I believe that `TimeCards` and `SalesReceipts` should be destroyed when the employee is destroyed.

The method of payment must also be changeable. Figure 18-6 implements this idea by using the STRATEGY pattern and deriving three different kinds of `PaymentMethod` classes. If an `Employee` object contains a

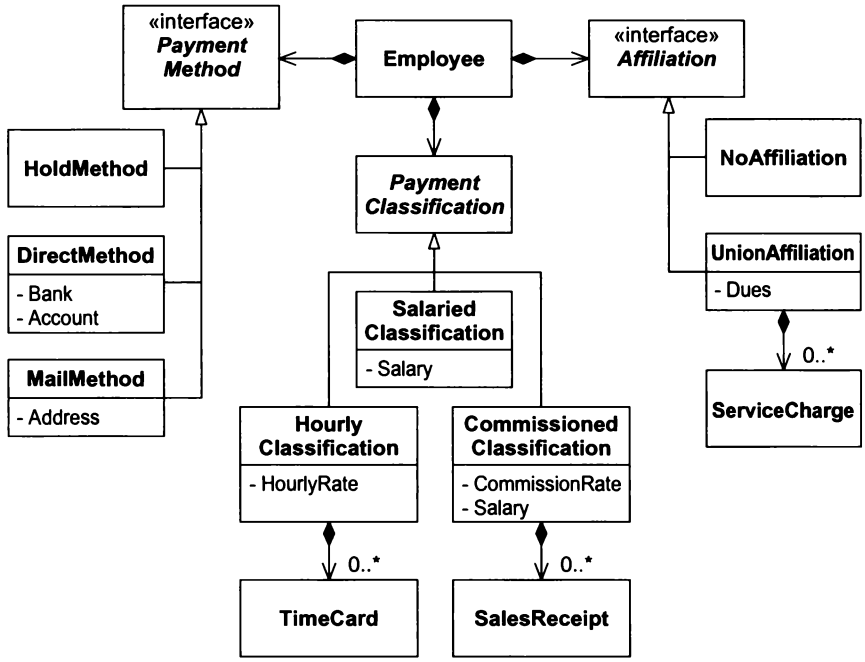


Figure 18-6 Revised Class Diagram for `Payroll -- The Core Model`

`MailMethod` object, the corresponding employee will have his paychecks mailed to him. The address to which the checks are mailed is recorded in the `MailMethod` object. If the `Employee` object contains a `DirectMethod` object, then his pay will be directly deposited into the bank account that is recorded in the `DirectMethod` object. If the `Employee` contains a `HoldMethod` object, his paychecks will be sent to the paymaster to be held for pickup.

Finally, Figure 18-6 applies the NULL OBJECT pattern to union membership. Each `Employee` object contains an `Affiliation` object, which has two forms. If the `Employee` contains a `NoAffiliation` object, then his pay is not adjusted by any organization other than the employer. However, if the `Employee` object contains a `UnionAffiliation` object, that employee must pay the dues and service charges that are recorded in that `UnionAffiliation` object.

This use of these patterns makes this system conform well to the Open-Closed Principle (OCP). The `Employee` class is closed against changes in payment method, payment classification, and union affiliation. New methods, classifications, and affiliations can be added to the system without affecting `Employee`.

Figure 18-6 is becoming our *core model* or architecture. It's at the heart of everything that the payroll system does. There will be many other classes and designs in the payroll application, but they will all be secondary to this fundamental structure. Of course, this structure is not cast in stone: It will be evolving along with everything else.

Payday

Use Case 7
Run the Payroll for Today

> Upon receiving the Payday transaction, the system finds all those employees that should be paid on the specified date. The system then determines how much they are owed and pays them according to their selected payment method.
>
> `Payday <date>`

Although it is easy to understand the intent of this use case, it is not so simple to determine what impact it has on the static structure of Figure 18-6. We need to answer several questions.

First, how does the `Employee` object know how to calculate its pay? Certainly if the employee is hourly, the system must tally up his time cards and multiply by the hourly rate. If the employee is commissioned, the system must tally up his sales receipts, multiply by the commission rate, and add the base salary. But where does this get done? The ideal place seems to be in the `PaymentClassification` derivatives. These objects maintain the records needed to calculate pay, so they should probably have the methods for determining pay. Figure 18-7 shows a collaboration diagram that describes how this might work.

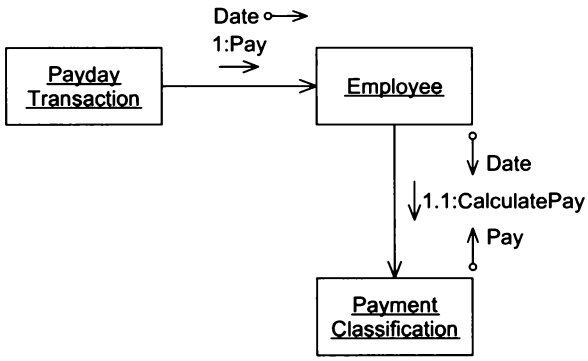


Figure 18-7 Calculating an Employee's Pay

When the `Employee` object is asked to calculate pay, it refers this request to its `PaymentClassification` object. The actual algorithm employed depends on the type of `PaymentClassification` that the `Employee` object contains. Figures 18-8 through 18-10 show the three possible scenarios.

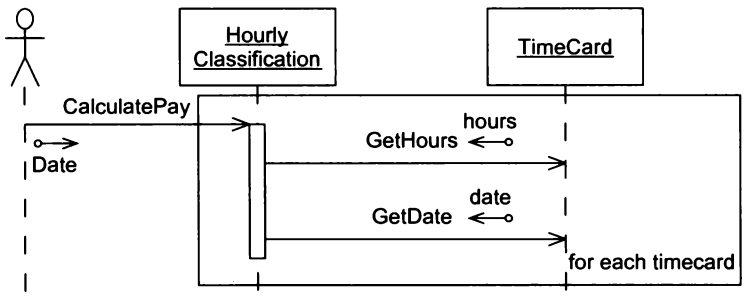


Figure 18-8 Calculating an Hourly Employee's Pay

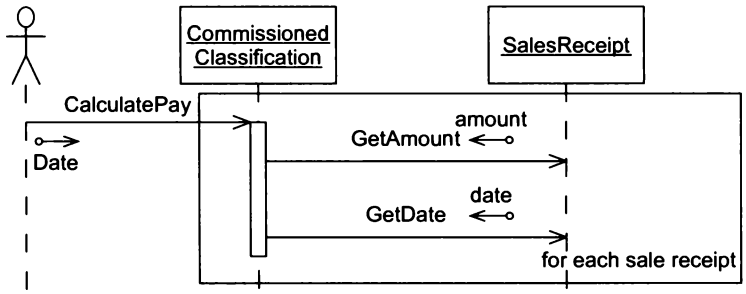


Figure 18-9 Calculating a Commissioned Employee's Pay

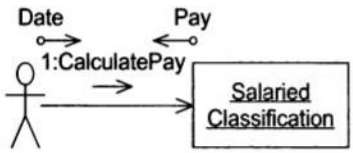
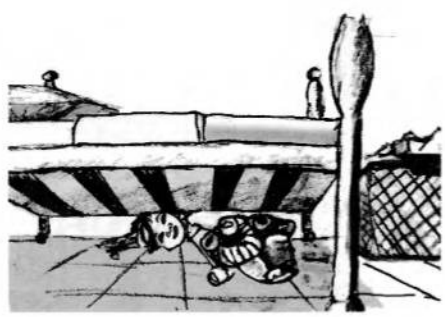


Figure 18-10 Calculating a Salaried Employee's Pay

Reflection: What Have We Learned?

We have learned that a simple use case analysis can provide a wealth of information and insights into the design of a system. Figures 18-6 through 18-10 came about by thinking about the use cases, that is, thinking about behavior.

Finding the Underlying Abstractions



To use the OCP effectively, we must hunt for abstractions and find those that underlie the application. Often these abstractions are not stated or even alluded to by the requirements of the application, or even the use cases. Requirements and use cases may be too steeped in details to express the generalities of the underlying abstractions.

What are the underlying abstractions of the Payroll application? Let's look again at the requirements. We see statements such as "Some employees work by the hour," "Some employees are paid a flat salary," and "Some [...] employees are paid a commission." This hints at the following generalization: "All employees are paid, but they are paid by different schemes." The abstraction here is that "All employees are paid." Our model of the `PaymentClassification` in Figures 18-7 through 18-10 expresses this abstraction nicely. Thus, this abstraction has already been found among our user stories by doing a very simple use-case analysis.

The Schedule Abstraction

Looking for other abstractions, we find "They are paid every Friday," "They are paid on the last working day of the month," and "They are paid every other Friday." This leads us to another generality: "All employees are paid according to some schedule." The abstraction here is the notion of the *schedule*. It should be possible to ask an `Employee` object whether a certain date is its payday. The use cases barely mention this. The requirements associate an employee's schedule with his payment classification. Specifically, hourly employees are paid weekly, salaried employees are paid monthly, and employees receiving commissions are paid biweekly; however, is this association essential? Might not the policy change one day so that employees could select a particular schedule or so that employees belonging to different departments or different divisions could have different schedules? Might not the schedule policy change independently of the payment policy? Certainly, this seems likely.

If, as the requirements imply, we delegated the issue of schedule to the `PaymentClassification` class, then our class could not be closed against issues of change in schedule. When we changed payment policy, we would also have to test schedule. When we changed schedules, we would also have to test payment policy. Both the OCP and the SRP would be violated.

An association between schedule and payment policy could lead to bugs in which a change to a particular payment policy caused incorrect scheduling of certain employees. Bugs like this may make sense to programmers, but they strike fear in the hearts of managers and users. They fear, and rightly so, that if schedules can be broken by a change to payment policy, then *any* change made *anywhere* might cause problems in *any* other unrelated part of the system. They fear that they cannot predict the effects of a change. When effects cannot be predicted, confidence is lost and the program assumes the status of "dangerous and unstable" in the minds of its managers and users.

Despite the essential nature of the schedule abstraction, our use-case analysis failed to give us any direct clues about its existence. To spot it required careful consideration of the requirements and an insight into the wiles of the user community. Overreliance on tools and procedures, and underreliance on intelligence and experience are recipes for disaster.

Figures 18-11 and 18-12 show the static and dynamic models for the schedule abstraction. As you can see, we've employed the STRATEGY pattern yet again. The `Employee` class contains the abstract `PaymentSchedule` class. There are three varieties of `PaymentSchedule` that correspond to the three known schedules by which employees are paid.

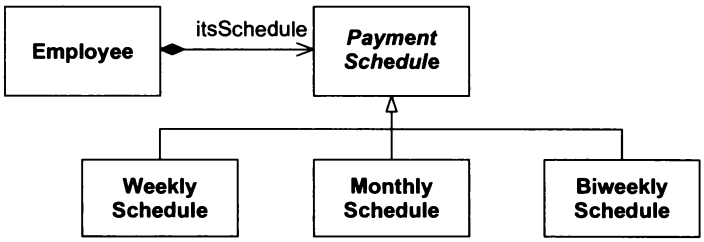


Figure 18-11 Static Model of a `Schedule` Abstraction

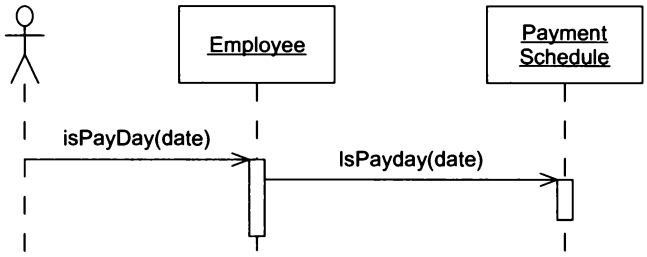


Figure 18-12 Dynamic Model of Schedule Abstraction

Payment Methods

Another generalization that we can make from the requirements is "All employees receive their pay by some method." The abstraction is the `PaymentMethod` class. Interestingly enough, this abstraction is already expressed in Figure 18-6.

Affiliations

The requirements imply that employees may have affiliations with a union; however, the union may not be the only organization that has a claim to some of an employee's pay. Employees might want to make automatic contributions to certain charities or have their dues to professional associations paid automatically. The generalization therefore becomes "The employee may be affiliated with many organizations that should be automatically paid from the employee's paycheck."

The corresponding abstraction is the `Affiliation` class that is shown in Figure 18-6. That figure, however, does not show the `Employee` containing more than one `Affiliation`, and it shows the presence of a `NoAffiliation` class. This design does not quite fit the abstraction we now think we need. Figures 18-13 and 18-14 show the static and dynamic models that represent the `Affiliation` abstraction.

The list of `Affiliation` objects has obviated the need to use the NULL OBJECT pattern for unaffiliated employees. Now, if the employee has no affiliation, his or her list of affiliations will simply be empty.

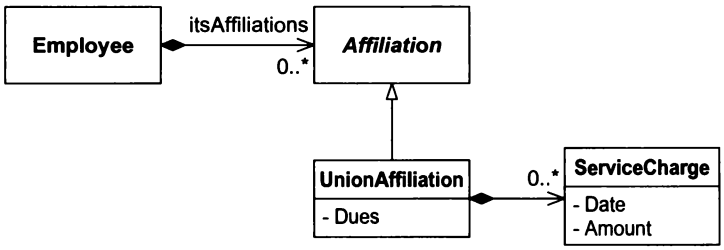


Figure 18-13 Static Structure of `Affiliation` Abstraction

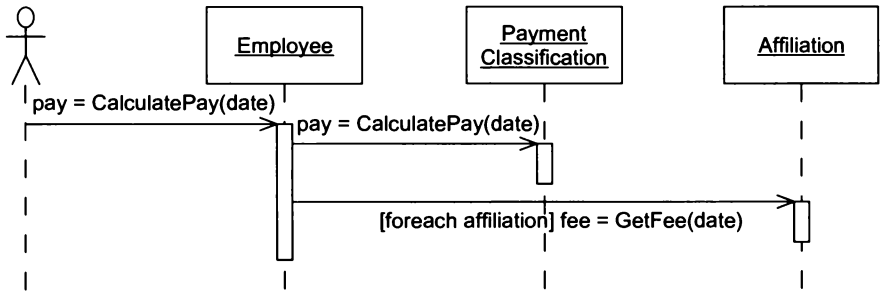


Figure 18-14 Dynamic Structure of `Affiliation` Abstraction

Conclusion

At the beginning of an iteration it is not uncommon to see the team assemble in front of a whiteboard and reason together about the design for the user stories that were selected for that iteration. Such a *quick design session* typically lasts less than an hour. The resulting UML diagrams, if any, may be left on the whiteboard, or erased. They are usually not commited to paper. The purpose of the session is to *start* the thinking process, and give the developers a common mental model to work from. The goal is *not* to nail down the design.

This chapter has been the textual equivalent to such a quick design Session.

Bibliography

1. Jacobson, Ivar. *Object-Oriented Software Engineering, A Use-Case-Driven Approach*. Wokingham, England: Addison–Wesley, 1992.

19

The Payroll Case Study: Implementation

It's long past time we started writing the code that supports and verifies the designs we've been spinning. I'll be creating that code in very small incremental steps, but I'll show it to you only at convenient points in the text. Don't let the fact that you only see fully formed snapshots of code mislead you into thinking that I wrote it in that form. In fact, between each batch of code you see, there will have been dozens of edits, compiles and test cases, each one making a tiny evolutionary change in the code.

You'll also see quite a bit of UML. Think of this UML as a quick diagram that I sketch on a whiteboard to show you, my pair partner, what I have in mind. UML makes a convenient medium for you and me to communicate by.

Figure 19-1 shows that we represent transactions as an abstract base class named `Transaction`, which has an instance method named `Execute()`. This is, of course, the COMMAND pattern. The implementation of the `Transaction` class is shown in Listing 19-1.

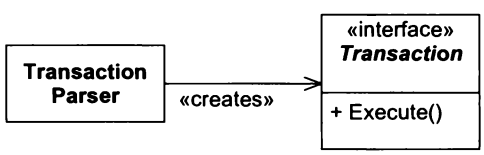


Figure 19-1 Transaction Interface

Listing 19-1

Transaction.h

```
#ifndef TRANSACTION_H
#define TRANSACTION_H

class Transaction
{
 public:
  virtual ~Transaction();
  virtual void Execute() = 0;
};

#endif
```

Adding Employees

Figure 19-2 shows a potential structure for the transactions that add employees. Note that it is within these transactions that the employees' payment schedule is associated with their payment classification. This is appropriate, since the transactions are contrivances instead of part of the core model. Thus, the core model is unaware of the association; the association is merely part of one of the contrivances and can be changed at any time. For example, we could easily add a transaction that allows us to change employee schedules.

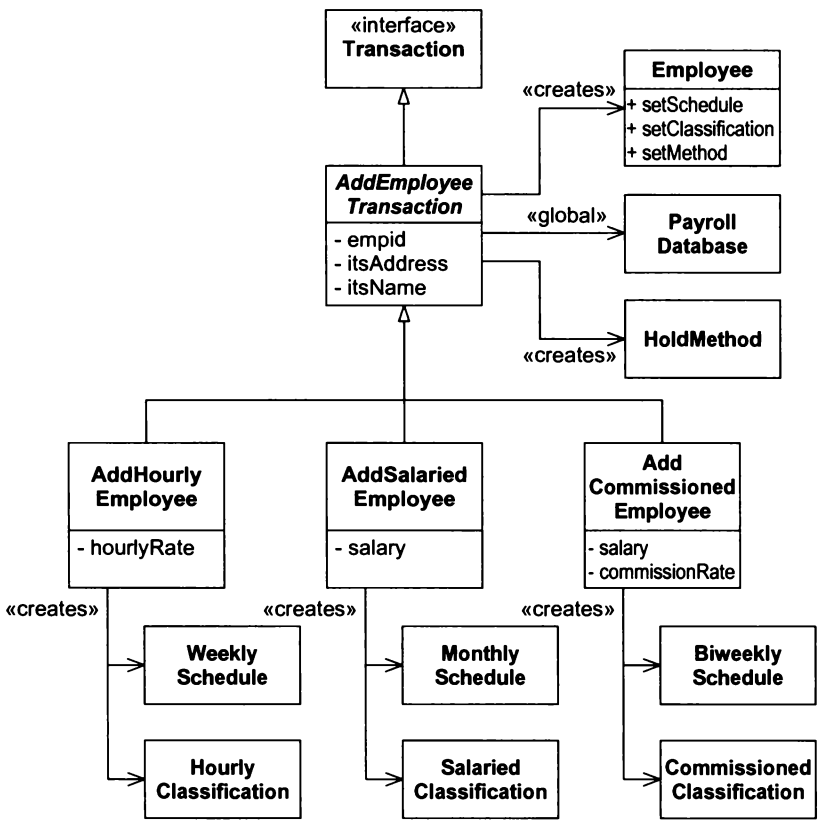


Figure 19-2 Static Model of `AddEmployeeTransaction`

Note, too, that the default payment method is to hold the paycheck with the paymaster. If an employee wants a different payment method, the change must be made with the appropriate `ChgEmp` transaction.

As usual, we begin writing code by writing tests first. Listing 19-2 is a test case that shows that the `AddSalariedTransaction` is working correctly. The code to follow will make that test case pass.

Listing 19-2

`PayrollTest::TestAddSalariedEmployee`

```
void PayrollTest::TestAddSalariedEmployee()
{
  int empId = 1;
  AddSalariedEmployee t(empId, "Bob", "Home", 1000.00);
  t.Execute();

  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert("Bob" == e->GetName());

  PaymentClassification* pc = e->GetClassification();
  SalariedClassification* sc = dynamic_cast<SalariedClassification*>(pc);
  assert(sc);

  assertEquals(1000.00, sc->GetSalary(), .001);
  PaymentSchedule* ps = e->GetSchedule();
  MonthlySchedule* ms = dynamic_cast<MonthlySchedule*>(ps);
  assert(ms);
  PaymentMethod* pm = e->GetMethod();
  HoldMethod* hm = dynamic_cast<HoldMethod*>(pm);
  assert(hm);
}
```

The Payroll Database

The `AddEmployeeTransaction` class uses a class called `PayrollDatabase`. This class maintains all the existing `Employee` objects in a `Dictionary` that are keyed by `empID`. It also maintains a `Dictionary` that maps union `memberIDs` to `empIDs`. The structure for this class appears in Figure 19-3. `PayrollDatabase` is an example of the FACADE pattern (page 173).

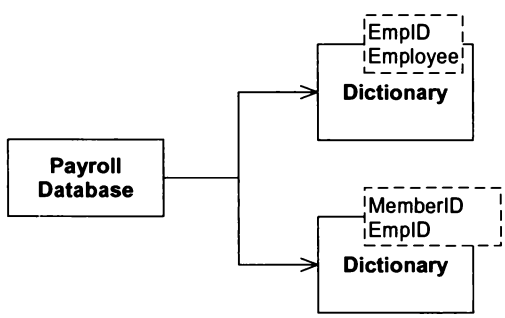


Figure 19-3 Static Structure of `PayrollDatabase`

Listings 19-3 and 19-4 show a rudimentary implementation of the `PayrollDatabase`. This implementation is meant to help us with our initial test cases. It does not yet contain the dictionary that maps member IDs to `Employee` instances.

Listing 19-3

PayrollDatabase.h

```
#ifndef PAYROLLDATABASE_H
#define PAYROLLDATABASE_H

#include <map>

class Employee;

class PayrollDatabase
{
 public:
  virtual ~PayrollDatabase();
  Employee* GetEmployee(int empId);
  void AddEmployee(int empid, Employee*);
  void clear() {itsEmployees.clear();}
 private:
  map<int, Employee*> itsEmployees;
};

#endif
```

Listing 19-4

PayrollDatabase.cpp

```
#include "PayrollDatabase.h"
#include "Employee.h"

PayrollDatabase GpayrollDatabase;

PayrollDatabase::~PayrollDatabase()
{
}

Employee* PayrollDatabase::GetEmployee(int empid)
{
  return itsEmployees[empid];
}

void PayrollDatabase::AddEmployee(int empid, Employee* e)
{
  itsEmployees[empid] = e;
}
```

In general, I consider database implementations to be details. Decisions about those details should be deferred as long as possible. Whether this particular database will be implemented with an RDBMS, flat files, or an OODBMS is irrelevant at this point. Right now, I'm just interested in creating the API that will provide database services to the rest of the application. I'll find appropriate implementations for the database later.

Deferring details about the database is an uncommon, but very rewarding, practice. Database decisions can usually wait until we have much more knowledge about the software and its needs. By waiting, we avoid the problem of putting too much infrastructure into the database. Rather, we implement just enough database facility for the needs of the application.

Using TEMPLATE METHOD to Add Employees

Figure 19-4 shows the dynamic model for adding an employee. Note that the `AddEmployeeTransaction` object sends messages to *itself* in order to get the appropriate `PaymentClassification` and `PaymentSchedule` objects. These messages are implemented in the derivatives of the `AddEmployeeTransaction` class. This is an application of the TEMPLATE METHOD pattern.

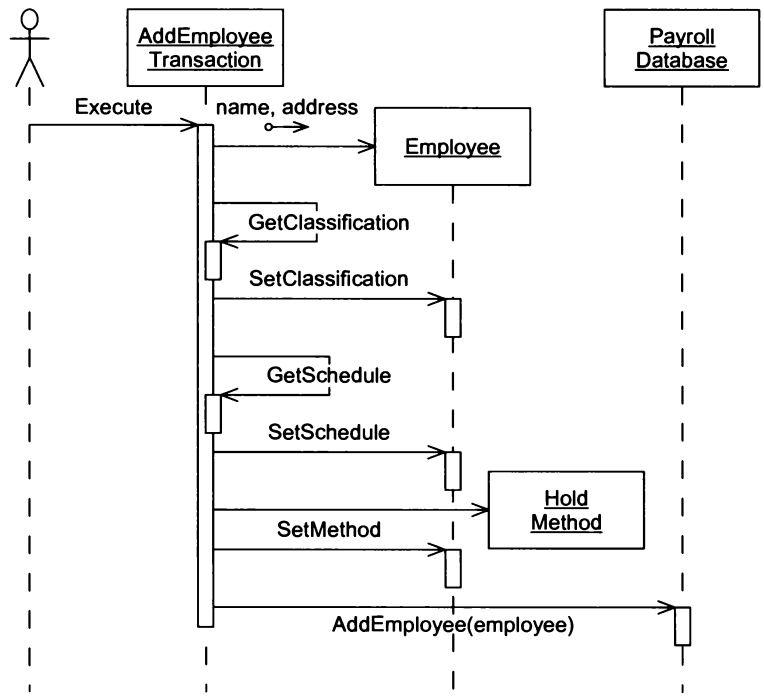


Figure 19-4 Dynamic Model for Adding an Employee

Listings 19-5 and 19-6 show the implementation of the TEMPLATE METHOD pattern in the `AddEmployeeTransaction` class. This class implements the `Execute()` method to call two pure virtual functions that will be implemented by derivatives. These functions, `GetSchedule()` and `GetClassification()`, return the `PaymentSchedule` and `PaymentClassification` objects that the newly created `Employee` needs. The `Execute()` method then binds these objects to the `Employee` and saves the `Employee` in the `PayrollDatabase`.

Listing 19-5

AddEmployeeTransaction.h

```
#ifndef ADDEMPLOYEETRANSACTION_H
#define ADDEMPLOYEETRANSACTION_H

#include "Transaction.h"
#include <string>

class PaymentClassification;
class PaymentSchedule;
```

```
class AddEmployeeTransaction : public Transaction
{
 public:
  virtual ~AddEmployeeTransaction();
  AddEmployeeTransaction(int empid, string name, string address);
  virtual PaymentClassification* GetClassification() const = 0;
  virtual PaymentSchedule* GetSchedule() const = 0;
  virtual void Execute();

 private:
  int itsEmpid;
  string itsName;
  string itsAddress;
};
#endif
```

Listing 19-6

AddEmployeeTransaction.cpp

```
#include "AddEmployeeTransaction.h"
#include "HoldMethod.h"
#include "Employee.h"
#include "PayrollDatabase.h"

class PaymentMethod;
class PaymentSchedule;
class PaymentClassification;

extern PayrollDatabase GpayrollDatabase;

AddEmployeeTransaction::~AddEmployeeTransaction()
{
}

AddEmployeeTransaction::
AddEmployeeTransaction(int empid, string name, string address)
  : itsEmpid(empid)
  , itsName(name)
  , itsAddress(address)
{
}

void AddEmployeeTransaction::Execute()
{
  PaymentClassification* pc = GetClassification();
  PaymentSchedule* ps = GetSchedule();
  PaymentMethod* pm = new HoldMethod();
  Employee* e = new Employee(itsEmpid, itsName, itsAddress);
  e->SetClassification(pc);
  e->SetSchedule(ps);
  e->SetMethod(pm);
  GpayrollDatabase.AddEmployee(itsEmpid, e);
}
```

Listings 19-7 and 19-8 show the implementation of the `AddSalariedEmployee` class. This class derives from `AddEmployeeTransaction` and implements the `GetSchedule()` and `GetClassification()` methods to pass back the appropriate objects to `AddEmployeeTransaction::Execute()`.

Listing 19-7

AddSalariedEmployee.h

```
#ifndef ADDSALARIEDEMPLOYEE_H
#define ADDSALARIEDEMPLOYEE_H

#include "AddEmployeeTransaction.h"

class AddSalariedEmployee : public AddEmployeeTransaction
{
 public:
  virtual ~AddSalariedEmployee();
  AddSalariedEmployee(int empid, string name,
                      string address, double salary);
  PaymentClassification* GetClassification() const;
  PaymentSchedule* GetSchedule() const;

 private:
  double itsSalary;
};
#endif
```

Listing 19-8

AddSalariedEmployee.cpp

```
#include "AddSalariedEmployee.h"
#include "SalariedClassification.h"
#include "MonthlySchedule.h"

AddSalariedEmployee::~AddSalariedEmployee()
{
}

AddSalariedEmployee::
AddSalariedEmployee(int empid, string name,
                    string address, double salary)
  : AddEmployeeTransaction(empid, name, address)
  , itsSalary(salary)
{
}


PaymentClassification*
AddSalariedEmployee::GetClassification() const
{
  return new SalariedClassification(itsSalary);
}

PaymentSchedule* AddSalariedEmployee::GetSchedule() const
{
  return new MonthlySchedule();
}
```

I leave the `AddHourlyEmployee` and `AddCommissionedEmployee` as exercises for the reader. Remember to write your test cases first.

Deleting Employees

Figures 19-5 and 19-6 present the static and dynamic models for the transactions that delete employees.

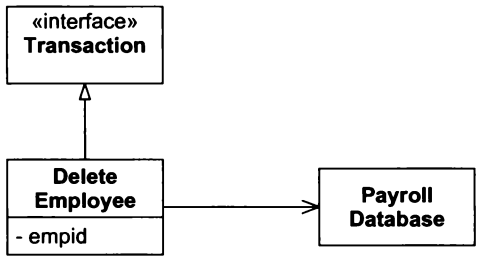


Figure 19-5 Static Model for `DeleteEmployee` Transaction

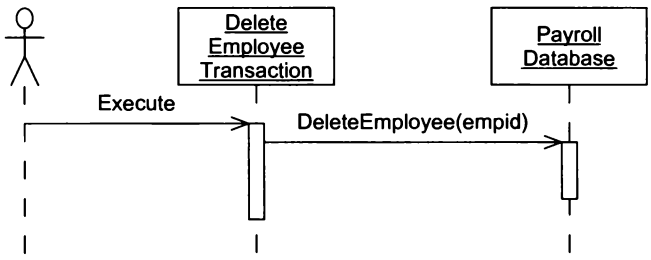


Figure 19-6 Dynamic Model for `DeleteEmployee` Transaction

Listing 19-9 shows the test case for deleting an employee. Listings 19-10 and 19-11 show the implementation of `DeleteEmployeeTransaction`. This is a very typical implementation of the COMMAND pattern. The constructor stores the data that the `Execute()` method eventually operates upon.

Listing 19-9

`PayrollTest::TestDeleteEmployee()`

```
void PayrollTest::TestDeleteEmployee()
{
  cerr << "TestDeleteEmployee" << endl;
  int empId = 3;
  AddCommissionedEmployee t(empId, "Lance", "Home", 2500, 3.2);
  t.Execute();
  {
    Employee* e = GpayrollDatabase.GetEmployee(empId);
    assert(e);
  }
  DeleteEmployeeTransaction dt(empId);
  dt.Execute();
  {
    Employee* e = GpayrollDatabase.GetEmployee(empId);
    assert(e == 0);
  }
}
```

Listing 19-10

DeleteEmployeeTransaction.h

```
#ifndef DELETEEMPLOYEETRANSACTION_H
#define DELETEEMPLOYEETRANSACTION_H

#include "Transaction.h"

class DeleteEmployeeTransaction : public Transaction
{
 public:
  virtual ~DeleteEmployeeTransaction();
  DeleteEmployeeTransaction(int empid);
  virtual void Execute();
 private:
  int itsEmpid;
};
#endif
```

Listing 19-11

DeleteEmployeeTransaction.cpp

```
#include "DeleteEmployeeTransaction.h"
#include "PayrollDatabase.h"

extern PayrollDatabase GpayrollDatabase;
DeleteEmployeeTransaction::~DeleteEmployeeTransaction()
{
}

DeleteEmployeeTransaction::DeleteEmployeeTransaction(int empid)
  : itsEmpid(empid)
{
}

void DeleteEmployeeTransaction::Execute()
{
  GpayrollDatabase.DeleteEmployee(itsEmpid);
}
```

Global Variables

By now you have noticed the `GpayrollDatabase` global. For decades, textbooks and teachers have been discouraging the use of global variables with good reason. Still, global variables are not intrinsically evil or harmful. This particular situation is an ideal choice for a global variable. There will only ever be one instance of the `PayrollDatabase` class, and it needs to be known by a very wide audience.

You might think that this could be better accomplished by using the SINGLETON or MONOSTATE patterns. It is true that these would serve the purpose. However, they do so by using global variables themselves. A SINGLETON or MONOSTATE is, by definition, a global entity. In this case I felt that a SINGLETON or MONOSTATE would smell of Needless Complexity. It's easier to simply keep the database instance in a global.

Time Cards, Sales Receipts, and Service Charges

Figure 19-7 shows the static structure for the transaction that posts time cards to employees. Figure 19-8 shows the dynamic model. The basic idea is that the transaction gets the `Employee` object from the `PayrollDatabase`, asks the `Employee` for its `PaymentClassification` object, and then creates and adds a `TimeCard` object to that `PaymentClassification`.

Notice that we cannot add `TimeCard` objects to general `PaymentClassification` objects; we can only add them to `HourlyClassification` objects. This implies that we must downcast the `PaymentClassification` object received from the `Employee` object to an `HourlyClassification` object. This is a good use for the `dynamic_cast` operator in C++, as shown later in Listing 19-15.

Listing 19-12 shows one of the test cases that verifies that time cards can be added to hourly employees. This test code simply creates an hourly employee and adds it to the database. Then it creates a `TimeCardTransaction` and invokes `Execute()`. Then it checks the employee to see if the `HourlyClassification` contains the appropriate `TimeCard`.

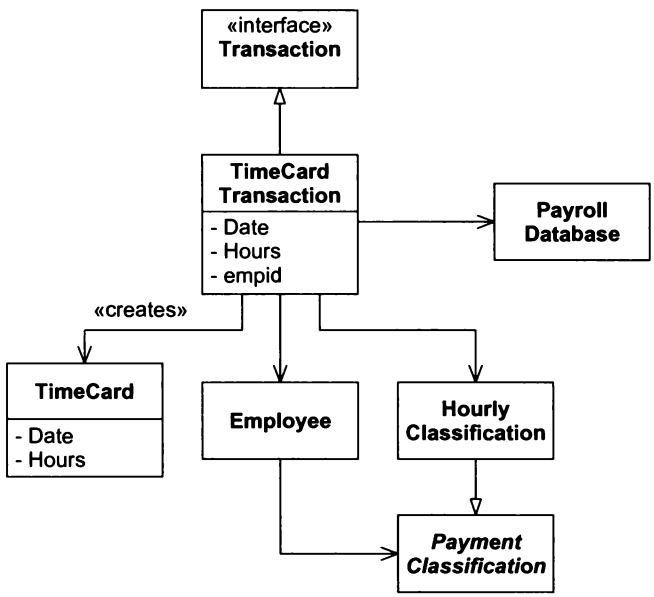


Figure 19-7 Static Structure of `TimeCardTransaction`

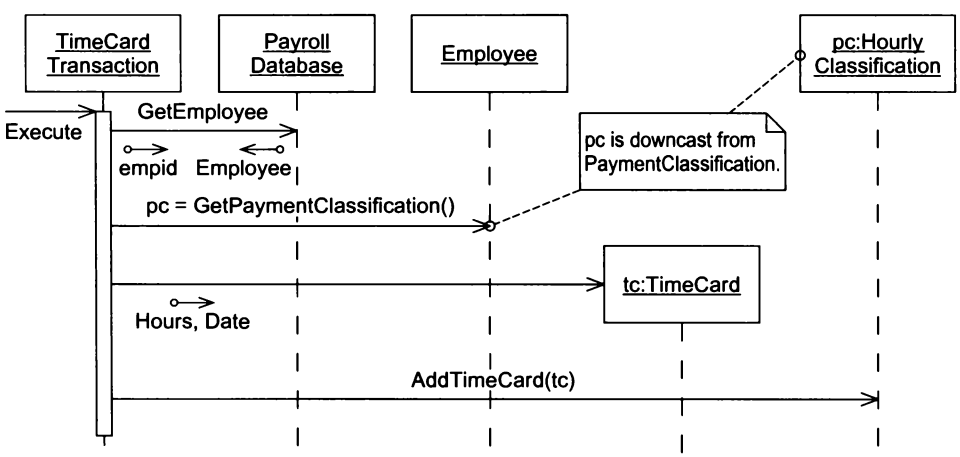


Figure 19-8 Dynamic Model for Posting a `TimeCard`

Listing 19-12

PayrollTest::TestTimeCardTransaction()

```
void PayrollTest::TestTimeCardTransaction()
{
  cerr << "TestTimeCardTransaction" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  TimeCardTransaction tct(20011031, 8.0, empId);
  tct.Execute();
  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert(e);
  PaymentClassification* pc = e->GetClassification();
  HourlyClassification* hc =
    dynamic_cast<HourlyClassification*>(pc);
  assert(hc);
  TimeCard* tc = hc->GetTimeCard(20011031);
  assert(tc);
  assertEquals(8.0, tc->GetHours());
}
```

Listing 19-13 shows the implementation of the `TimeCard` class. There's not much to this class right now. It's just a data class. Notice that I am using a long integer to represent dates. I'm doing this because I don't have a convenient `Date` class. I'm probably going to need one pretty soon, but I don't need it now. I don't want to distract myself from the task at hand, which is to get the current test case working. Eventually I will write a test case that will require a true `Date` class. When that happens, I'll go back and retrofit it into `TimeCard`.

Listing 19-13

TimeCard.h

```
#ifndef TIMECARD_H
#define TIMECARD_H

class TimeCard
{
 public:
  virtual ~TimeCard();
  TimeCard(long date, double hours);
  long GetDate() {return itsDate;}
  double GetHours() {return itsHours;}
 private:
  long itsDate;
  double itsHours;
};
#endif
```

Listings 19-14 and 19-15 show the implementation of the `TimeCardTransaction` class. Note the use of simple string exceptions. This is not particularly good long-term practice, but it suffices this early in development. After we get some idea of what the exceptions really ought to be, we can come back and create meaningful exception classes. Note also that the `TimeCard` instance is created only when we are sure we aren't going to throw an exception, so the throwing of the exception can't leak memory. It's very easy to create code that leaks memory or resources when throwing exceptions, so be careful. [1]

1. And run, don't walk, to buy *Exceptional C++* and *More Exceptional C++*, by Herb Sutter. These two books will save you much anguish, wailing, and gnashing of teeth over exceptions in C++.